



 Product Requirements Document

ChromaLens

Pixel-Level Image Color Analyzer

VERSION

2.0 — Final

DATE

June 18, 2026

AUTHOR

Komal Barhate

STATUS

Final

AUDIENCE

Engineers, QA, Design

PROTOTYPE

ChromaLens v1.0

Table of Contents

01	Background & Problem Statement
02	Use Case — Detailed Scenario Walkthrough
03	User Personas
04	Goals & Success Metrics
05	Scope — In & Out of v1.0
06	Expected Inputs
07	Color-Mapping Algorithm (Full Specification)
08	Desired Output
09	Functional Requirements (F-01 to F-34)
10	Non-Functional Requirements
11	Edge Cases & Error Handling
12	Acceptance Criteria (AC-01 to AC-17)
13	Technical Architecture
14	Future Roadmap
15	Appendix — Worked Examples

01 — Background & Problem Statement

1.1 Context

In design, marketing, and product workflows, teams frequently need to understand the **color composition** of images — for brand audits, packaging reviews, social media consistency, or content moderation. Currently, professionals rely on one of two inadequate approaches:

- **Manual inspection** — subjective, time-consuming, and inconsistent across team members.
- **Dominant color palette tools** — return raw hex codes (e.g. `#4A2C1D`) which are meaningless to non-technical users and don't map to human-understandable color names.

1.2 The Gap

There is no lightweight, zero-installation, privacy-preserving tool that:

- Analyzes images at the **pixel level** (every single pixel, not just a sample),
- Maps results to **human-readable color categories** (e.g. "Navy → Blue", "Burgundy → Red"),
- Displays results as clear **percentage charts**, and
- Works **instantly in a browser** — no backend, no sign-up, no API cost.

1.3 The Solution

ChromaLens is a browser-based tool that accepts an image, reads every pixel using the HTML5 Canvas API, classifies each pixel into one of **11 universal color categories**, and displays an interactive color percentage breakdown as charts, swatches, and an animated summary panel. Additionally, users can tap any point on the image to identify that exact pixel's color in real time.

⚡ KEY DIFFERENTIATOR

100% client-side processing — zero pixel data is ever transmitted to any server.
The tool works offline once loaded.

02 — Use Case

"As a designer, marketer, or product manager, I want to upload any image and instantly see what percentage of it is each primary color, so I can make informed decisions about visual branding, content, and design."

2.1 Detailed Scenario Walkthrough

Step 1 — Upload

The user opens ChromaLens in their browser. They see a drag-and-drop zone with a "Choose Image" button. They either drag an image file onto the zone or click to open a file picker. Accepted formats: PNG, JPG, WEBP.

Step 2 — Processing

A loading overlay appears ("Analyzing pixels...") with an animated spinner. The tool reads every pixel of the image using the Canvas `getImageData` API, converts each pixel's RGB values to HSL, and runs it through a 14-rule classification decision tree to assign it to one of 11 color buckets.

Step 3 — Results

Within 1–2 seconds, the results section replaces the upload zone, showing:

- A **preview** of the uploaded image (with a crosshair cursor)
- An **Analysis Summary** panel with animated color percentage bars
- A **Doughnut chart** of color distribution
- A **Horizontal bar chart** of percentages by color
- A **Color legend** grid with swatches and percentages

Step 4 — Interactive Color Picker

The user taps/clicks any point on the image preview. A floating tooltip pops up (with a spring animation) showing the exact color category (e.g. "Green"), the actual hex value (e.g. #3A8C45), and a colored swatch. A pulsing dot appears at the tap point.

Step 5 — Export or Reset

The user can download the doughnut or bar chart as a PNG file, or click "Analyze Another" to return to the upload state.

03 — User Personas

Persona	Role	Goal	How They Use ChromaLens
Priya	Senior Designer	Audit whether campaign images match brand color guidelines (green, white, neutral grays only)	Uploads each image, screenshots the color breakdown chart, shares in design review presentations
Ravi	Digital Marketing Manager	Ensure social posts are color-consistent with brand palette before publishing	Drags & drops the post image, checks Green ≥ 40%, White ≥ 20%
Ananya	Frontend Developer	Build prototype for stakeholder review; later integrate the algorithm into a production pipeline	Uses tool as working prototype; reads PRD for algorithm spec to implement in backend

04 — Goals & Success Metrics

Goal	Metric	Target
Accurate classification	% of pixels correctly categorized vs. human labeling	≥ 95%
Speed	Upload → results displayed	< 2 seconds (5 MB, mid-range laptop)
Zero friction	Steps to first result	≤ 2 steps (upload → see results)
Export	Downloaded charts render correctly	100% success rate
Privacy	Pixel data transmitted externally	0 bytes

05 — Scope

In Scope — v1.0

- Single image upload (file dialog + drag-and-drop)
- Pixel-level analysis via HTML5 Canvas `getImageData` API
- Classification into exactly 11 color categories
- Doughnut chart, horizontal bar chart, swatch legend, animated summary panel
- Interactive pixel color picker (click image → see color name + hex)
- Download charts as PNG files
- 100% client-side — no server required
- Responsive layout (desktop + mobile ≥ 375px)

Out of Scope — v1.0

- Batch image processing
- Live camera / webcam analysis
- Custom color bucket definitions

- Dominant hex palette extraction (top-N exact hex values)
- Figma / Adobe plugin integration
- REST API endpoint
- User accounts or history

06 — Expected Inputs

6.1 Image File

Attribute	Specification
Delivery Method	File picker dialog OR drag-and-drop onto upload zone
Accepted Formats	JPEG (.jpg, .jpeg), PNG (.png), WEBP (.webp)
Maximum File Size	20 MB
Minimum Dimensions	1 × 1 pixel
Maximum Dimensions	No hard limit; images are downscaled internally for performance
Color Space	RGB or RGBA; CMYK not supported (browser auto-converts JPEG/PNG)
Transparency	Fully transparent pixels (alpha < 20/255) are excluded from analysis

6.2 Interactive Input — Color Picker

Attribute	Specification
Trigger	Single tap/click on the preview image after analysis completes
Valid Area	Only within the actual image bounds (letterbox/pillarbox margins excluded)
Output	Color category name + exact pixel hex value displayed in tooltip

6.3 Input Validation Rules

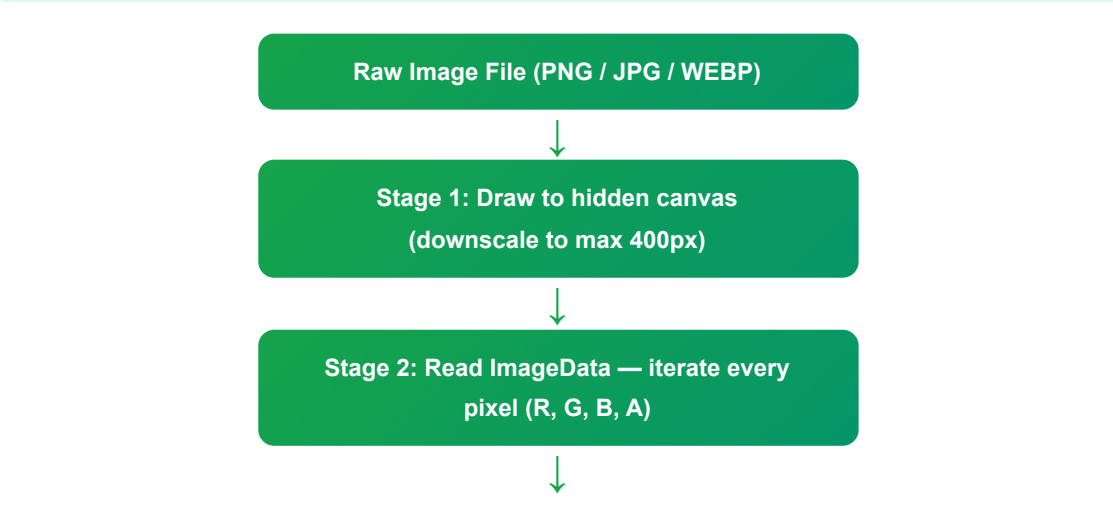
Condition	Expected Behavior
File type not image/*	Browser <code>accept</code> attribute restricts picker; invalid drag-drop silently ignored
Image fails to decode	<code>onerror</code> handler dismisses overlay; shows friendly alert
Transparent pixel tapped	No tooltip shown; do nothing
Click outside image area	No tooltip shown; letterbox margins are excluded

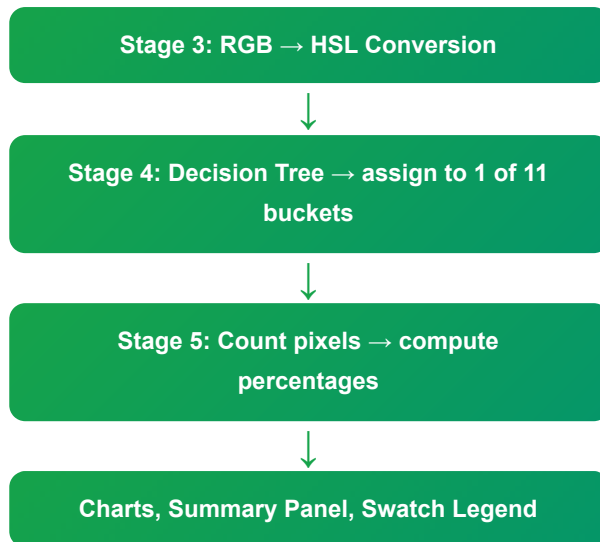
07 — Color-Mapping Algorithm

DEVELOPER NOTE

This section is the authoritative specification. A developer should be able to implement an identical classification engine from this section alone without referring to any other source.

7.1 Pipeline Overview





7.2 Stage 1 — Image Downscaling

For performance, the image is drawn onto a hidden `<canvas>` downscaled so that the **longest dimension does not exceed 400px**, preserving aspect ratio.

```
scale      = min(1, 400 / max(naturalWidth, naturalHeight))
canvasWidth = round(naturalWidth * scale)
canvasHeight = round(naturalHeight * scale)
ctx.drawImage(img, 0, 0, canvasWidth, canvasHeight)
```

💡 WHY 400PX?

This limits pixel count to $\leq 160,000$, keeping analysis under 100ms on most devices while maintaining statistical accuracy. Color distribution is not materially affected by this downscale.

7.3 Stage 2 — Pixel Iteration

```
imageData = ctx.getImageData(0, 0, canvasWidth, canvasHeight)
data = imageData.data // Uint8ClampedArray - RGBA interleaved

for i = 0 to data.length step 4:
    R = data[i] // Red 0-255
    G = data[i+1] // Green 0-255
    B = data[i+2] // Blue 0-255
    A = data[i+3] // Alpha 0-255

    if A < 20: continue // skip fully transparent pixels

    label = classifyPixel(R, G, B)
    counts[label]++
```

7.4 Stage 3 — RGB to HSL Conversion

All pixel values are converted from **RGB (0–255)** to **HSL** — Hue (0–360°), Saturation (0–100%), Lightness (0–100%) — using the standard conversion formula:

```
r' = R / 255,    g' = G / 255,    b' = B / 255

cmax = max(r', g', b')
cmin = min(r', g', b')
Δ     = cmax - cmin

— LIGHTNESS —
L = (cmax + cmin) / 2

— SATURATION —
if Δ == 0:      S = 0    (achromatic)
else if L < 0.5: S = Δ / (cmax + cmin)
else:          S = Δ / (2 - cmax - cmin)

— HUE —
if Δ == 0:      H = 0    (achromatic)
else if cmax == r': H = ((g' - b') / Δ) mod 6 × 60
else if cmax == g': H = ((b' - r') / Δ + 2) × 60
else if cmax == b': H = ((r' - g') / Δ + 4) × 60

if H < 0: H += 360

Final output: L = L×100,  S = S×100  (both in range 0–100)
```

💡 WHY HSL?

HSL separates color (Hue) from vividness (Saturation) and brightness (Lightness), mirroring human color perception. This makes achromatic detection and hue-wheel classification natural and accurate.

7.5 Stage 4 — Classification Decision Tree

The following 14 rules are evaluated **top-to-bottom**. The **first matching rule wins**. All range endpoints are inclusive unless noted with `(` or `)`.

RULE 1 — WHITE (ACHROMATIC)

`if L ≥ 88 → return "White"`

Catches: ivory, cream, off-white, snow, eggshell

RULE 2 — BLACK (ACHROMATIC)

`if L ≤ 12 → return "Black"`

Catches: jet black, very dark charcoal, near-black navy

RULE 3 — GRAY (ACHROMATIC)

`if S < 16 → return "Gray"`

Catches: silver, ash, slate, all near-neutral mid-tones regardless of hue

RULE 4 — BROWN (PRIMARY)

`if H ∈ [0, 40] AND S ∈ [15, 60] AND L ∈ [15, 46] → return "Brown"`

Warm hue + moderate saturation + dark-ish = earth tone. Catches: tan, sienna, chocolate, saddle brown

RULE 5 — BROWN (ORANGE-BROWN EXTENSION)

`if H ∈ (10, 44] AND S ∈ [15, 55] AND L ∈ (22, 44] → return "Brown"`

Catches: terracotta, rust tones, warm orange-browns Rule 4 misses

RULE 6 — PINK (HIGH-LIGHTNESS REDDISH)

`if (H ≥ 315 OR H ≤ 14) AND L ≥ 60 AND S ≥ 15 → return "Pink"`

Reddish hue too light to be Red. Catches: rose, hot pink, salmon, coral

RULE 7 — PINK (DESATURATED BLUSH)

if $(H \geq 315 \text{ OR } H \leq 14) \text{ AND } S < 50 \text{ AND } L \geq 46 \rightarrow \text{return "Pink"}$

Low-saturation reddish-light. Catches: blush, pastel pink, light skin tones

RULE 8 — RED (HUE WHEEL)

if $H \leq 14 \text{ OR } H \geq 345 \rightarrow \text{return "Red"}$

Catches: scarlet, crimson, burgundy, maroon, fire red, dark red

RULE 9 — ORANGE (HUE WHEEL)

if $H \in (14, 44] \rightarrow \text{return "Orange"}$

Catches: amber, tangerine, pumpkin, fire orange

RULE 10 — YELLOW (HUE WHEEL)

if $H \in (44, 72] \rightarrow \text{return "Yellow"}$

Catches: gold, lemon, mustard, khaki, light yellow

RULE 11 — GREEN (HUE WHEEL)

if $H \in (72, 165] \rightarrow \text{return "Green"}$

Catches: lime, olive, forest, emerald, teal, sage, cyan-green

RULE 12 — BLUE (HUE WHEEL)

if $H \in (165, 265] \rightarrow \text{return "Blue"}$

Catches: navy, sky blue, cobalt, indigo, cerulean, cyan-blue

RULE 13 — PURPLE (HUE WHEEL)

if $H \in (265, 314] \rightarrow \text{return "Purple"}$

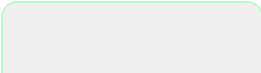
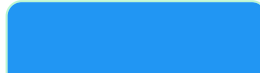
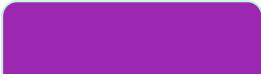
Catches: violet, lavender, mauve, plum, magenta-purple

RULE 14 — PINK (HUE WHEEL FALLBACK)

if $H \in (314, 360) \rightarrow \text{return "Pink"}$

Catches: remaining pinkish-magenta hues not caught by Rules 6–7

7.6 The 11 Color Categories

 White #F0F0F0 L ≥ 88%	 Black #212121 L ≤ 12%	 Gray #9E9E9E S < 16%	 Red #E53935 H ≤ 14° / ≥ 345°
 Orange #FF6D00 H 15–44°	 Yellow #F5E642 H 45–72°	 Green #4CAF50 H 73–165°	 Blue #2196F3 H 166–265°
 Purple #9C27B0 H 266–314°	 Pink #F06292 H 315–360° / high-L reds	 Brown #795548 Warm, dark, muted	

7.7 Stage 5 — Percentage Calculation

```
totalPixels = count of pixels where alpha ≥ 20

for each category C:
    percentage(C) = counts[C] / totalPixels × 100

Results sorted descending by pixel count before display.
Sum of all percentages ≈ 100% (±0.1% due to floating-point rounding).
```

7.8 Color Picker — Pixel Coordinate Mapping

When the user clicks the displayed image (rendered with `object-fit: contain`), the click must be mapped to canvas coordinates:

```

// Step 1: Get element dimensions
elementW = img.clientWidth
elementH = img.clientHeight
naturalW = img.naturalWidth
naturalH = img.naturalHeight

// Step 2: Compute contained image rect (accounting for letterboxing)
scale      = min(elementW / naturalW, elementH / naturalH)
renderedW = naturalW * scale
renderedH = naturalH * scale
offsetX    = (elementW - renderedW) / 2
offsetY    = (elementH - renderedH) / 2

// Step 3: Map click to image-relative coordinates
relX = clickX - offsetX
relY = clickY - offsetY
if relX < 0 or relY < 0 or relX > renderedW or relY > renderedH: ignore

// Step 4: Map to canvas pixel
canvasX = clamp(round(relX / renderedW * canvasWidth), 0, canvasWidth-1)
canvasY = clamp(round(relY / renderedH * canvasHeight), 0, canvasHeight-1)

// Step 5: Read and classify
[R,G,B,A] = ctx.getImageData(canvasX, canvasY, 1, 1).data
if A < 20: ignore
colorName = classifyPixel(R, G, B)
hex = "#" + R.hex + G.hex + B.hex (uppercase, zero-padded)

```

08 — Desired Output

8.1 Visual Outputs

Output	Description	Key Details
Analysis Summary Panel	Animated percentage bar per color	Only categories $\geq 0.1\%$ shown; bars animate 0% → target on load; dominant color = 100% bar width (relative scale)
Doughnut Chart	Color distribution ring chart	All 11 categories; hover tooltip shows category + %; animated scale-in 1s; cutout 58%
Horizontal Bar Chart	Percentage breakdown by color	Sorted descending; x-axis labeled %; hover tooltip shows %; animated fill 0.9s
Color Legend Grid	Swatch + name + % for each category	All 11 categories shown including 0.00%; sorted by % descending
Image Preview	Uploaded image display	object-fit: contain; crosshair cursor; caption: filename + "100.0% analysed"
Color Picker Tooltip	Real-time pixel color on tap	Category name (bold) + hex (monospace) + 42×42px swatch; spring animation; auto-dismisses 3s; pulse dot at click point

8.2 Downloadable Outputs

Button	Filename	Contents
↓ Download Pie Chart	color-distribution-doughnut.png	Doughnut chart canvas exported via <code>toDataURL('image/png')</code>
↓ Download Bar Chart	color-distribution-bar.png	Bar chart canvas exported via <code>toDataURL('image/png')</code>

8.3 Data Output Format (Future API Reference)

```
{
  "filename": "photo.jpg",
  "totalPixelsAnalyzed": 98432,
  "results": [
    { "name": "Green", "hex": "#4CAF50", "count": 41241, "percentage":
"41.90" },
    { "name": "White", "hex": "#F0F0F0", "count": 28710, "percentage":
"29.17" },
    { "name": "Blue", "hex": "#2196F3", "count": 15003, "percentage":
"15.24" },
    { "name": "Gray", "hex": "#9E9E9E", "count": 8801, "percentage":
"8.94" },
    { "name": "Black", "hex": "#212121", "count": 3200, "percentage":
"3.25" },
    { "name": "Yellow", "hex": "#FFEB3B", "count": 1021, "percentage":
"1.04" },
    { "name": "Orange", "hex": "#FF9800", "count": 0, "percentage":
"0.00" }
  ]
}
```


09 — Functional Requirements

9.1 Upload & Input

ID	Requirement
F-01	The system SHALL accept image upload via a file browser dialog
F-02	The system SHALL accept image upload via drag-and-drop onto the designated zone
F-03	The file input element SHALL restrict accepted files to <code>image/*</code>
F-04	Clicking anywhere on the upload card (except the label/input) SHALL open the file dialog
F-05	Clicking the "Choose Image" label SHALL open the file dialog exactly once (no double-trigger)
F-06	A visual drag-over state SHALL be applied to the upload zone during drag

9.2 Processing

ID	Requirement
F-07	A loading overlay SHALL appear immediately when a file is selected
F-08	The overlay SHALL contain an animated spinner and the text "Analyzing pixels..."
F-09	The image SHALL be drawn to a hidden <code><canvas></code> downscaled to max 400px longest dimension
F-10	Every non-transparent pixel (alpha ≥ 20) SHALL be read and classified
F-11	Classification SHALL follow the RGB→HSL→Decision Tree algorithm in Section 7
F-12	The overlay SHALL ALWAYS be dismissed after analysis — whether success or failure

9.3 Results Display

ID	Requirement
F-13	Results SHALL replace the upload zone with a smooth fade-in animation
F-14	Results SHALL include: image preview, analysis summary, doughnut chart, bar chart, swatch legend
F-15	The analysis summary SHALL show all categories with $\geq 0.1\%$ pixels
F-16	Percentage bars SHALL animate from 0% to their target width on entry
F-17	The doughnut chart SHALL animate on first render
F-18	The bar chart SHALL animate on first render
F-19	Chart tooltips SHALL display percentage values on hover

9.4 Interactive Color Picker

ID	Requirement
F-20	After analysis, the preview image SHALL display a crosshair cursor
F-21	A "Tap image to identify color" hint badge SHALL appear over the image
F-22	Clicking/tapping any point on the image SHALL identify the pixel color
F-23	A pulsing dot SHALL appear at the exact click point, colored with the pixel's actual color
F-24	A floating tooltip SHALL appear showing: category name, hex code, color swatch
F-25	The tooltip SHALL NOT appear for clicks in letterbox/pillarbox margins
F-26	The tooltip SHALL NOT appear for transparent pixels (alpha < 20)
F-27	The tooltip SHALL automatically dismiss after 3 seconds
F-28	Each new click SHALL reset and replay the animation fresh
F-29	The tooltip SHALL flip its position to stay within the viewport on edge clicks

9.5 Export & Reset

ID	Requirement
F-30	"Download Pie Chart" SHALL export the doughnut chart canvas as a PNG file
F-31	"Download Bar Chart" SHALL export the bar chart canvas as a PNG file
F-32	"Analyze Another" SHALL reset the tool to the upload state
F-33	On reset, the color picker tooltip and dot SHALL be cleared
F-34	On reset, the file input value SHALL be cleared so the same file can be re-selected

10 — Non-Functional Requirements

Category	Requirement
Performance	Analysis completes in < 2 seconds for a 5 MB image on a mid-range laptop (Chrome)
Privacy	Zero pixel data transmitted to any server; all processing is client-side
Compatibility	Chrome 90+, Firefox 90+, Safari 15+, Edge 90+
Responsiveness	Fully usable on screens ≥ 375px wide (mobile-friendly layout)
Accessibility	Chart tooltips provide text equivalents of visual data
Offline	Tool works without internet once the page is loaded (except Google Fonts CDN)
No Dependencies	No server, no database, no API keys, no sign-up required
Security	No external image URLs accepted; file upload only (prevents CORS pixel-read issues)

11 — Edge Cases & Error Handling

Scenario	Expected Behavior
All-white image	100% White; all other buckets 0.00%
All-black image	100% Black
Grayscale image	Distributed across White, Gray, and Black only
PNG with full transparency	0 pixels analysed → error message shown
PNG with partial transparency	Only opaque pixels ($\alpha \geq 20$) counted; total based on opaque count
Very small image (1×1 px)	Analyzed normally; 100% in the one pixel's bucket
Very large image (50 MP)	Downscaled to 400px; analysis completes within performance target
Click outside image area	No tooltip or dot shown (letterbox margin excluded)
Click on transparent pixel	No tooltip or dot shown
Runtime error during analysis	Overlay dismissed; alert: "Could not analyze image. Please try a different file."
Image fails to load (onerror)	Overlay dismissed; alert: "Failed to load the image. Please try a different file."
Same file uploaded twice	fileInput.value cleared on reset; change event fires normally

12 — Acceptance Criteria

✓ **AC-01** File dialog upload triggers analysis

✓ **AC-02** Drag-and-drop upload triggers analysis

✓ **AC-03** Loading overlay appears immediately on selection

✓ **AC-04** Overlay always dismisses (success or failure)

✓ **AC-05** All 11 categories present in charts

✓ **AC-06** Percentages sum to 100.00% ± 0.10%

✓ **AC-07** Navy pixel → "Blue" category

✓ **AC-08** Burgundy pixel → "Red" category

✓ **AC-09** Ivory pixel → "White" category

✓ **AC-10** Transparent pixels excluded from analysis

✓ **AC-11** Color picker shows correct category on tap

✓ **AC-12** Hex in tooltip matches getImageData result

✓ **AC-13** Tooltip flips at screen edges

✓ **AC-14** Tooltip and dot dismiss after 3 seconds

✓ **AC-15** Download Pie Chart saves correct PNG

✓ **AC-16** Download Bar Chart saves correct PNG

✓ **AC-17** Reset returns to upload; same file re-selectable

13 — Technical Architecture

User's Browser (Client-Side Only)	
index.html	analyzer.js
• Upload zone • Results section • Tooltip element • Picker dot	• <code>classifyPixel(r,g,b)</code> - <code>rgbToHsl()</code> - Decision Tree Rules 1-14 • <code>analyzeImage(imgEl)</code> - Draw to hidden canvas - <code>getImageData()</code> - Iterate all pixels - Count 11 buckets • <code>renderCharts()</code> - Chart.js doughnut + bar • <code>showColorTooltip()</code> - <code>getContainedImageRect()</code> - Spring animation + dot
style.css	
• Light green/white theme • Tooltip styles • Pulse dot anim • Glassmorphism	
External CDN (loaded once):	
• Chart.js 4.4.0 (jsdelivr.net) • Google Fonts: Inter + Space Grotesk	
No backend. No database. No API keys.	
Zero data leaves the browser.	

File Structure

ChromaLens/	
├ index.html	- Page structure, upload zone, results, tooltip elements
├ style.css	- Green/white light theme, animations, color picker styles
├ analyzer.js	- Classification engine, chart rendering, color picker logic
├ ChromaLens_PRD.pdf	- This document
└ README.md	- Project overview and setup instructions

14 — Future Roadmap

Feature	Version	Priority	Notes
Batch processing (4+ images)	v2.0	High	Process multiple images; compare charts side-by-side
Dominant hex palette	v2.0	High	Extract top 5 exact pixel hex values alongside categories
Custom color buckets	v2.0	Medium	User defines own brand colors; tool maps to those
Color picker history	v2.0	Medium	Log last 5 tapped pixel colors in a side panel
REST API endpoint	v2.1	Medium	POST /analyze → returns JSON breakdown
Figma plugin	v3.0	Low	Analyze selected frames directly inside Figma
Threshold configuration	v3.0	Low	User-configurable L/S sliders for achromatic detection
Camera capture	v3.0	Low	Analyze live webcam frame

15 — Appendix: Worked Examples

Example A — Dark Red RGB(139, 0, 0)

Step 1 – RGB → HSL:

$r' = 0.545$, $g' = 0$, $b' = 0$
 $c_{\max} = 0.545$, $c_{\min} = 0$, $\Delta = 0.545$
 $L = 27.3\%$, $S = 100\%$, $H = 0^\circ$

Step 2 – Decision Tree:

Rule 1: $L=27.3\% < 88 \rightarrow \text{skip}$
Rule 2: $L=27.3\% > 12 \rightarrow \text{skip}$
Rule 3: $S=100\% \geq 16 \rightarrow \text{skip}$
Rule 4: $S=100\% > 60 \rightarrow \text{skip}$ (out of $[15, 60]$ range)
Rule 6: $L=27.3\% < 60 \rightarrow \text{skip}$
Rule 8: $H=0^\circ \leq 14 \rightarrow \checkmark \text{ RED}$

Result: RGB(139,0,0) → RED $\checkmark$ (Dark Red correctly maps to Red)

Example B — Navy Blue RGB(0, 0, 128)

Step 1 – RGB → HSL:

$r' = 0$, $g' = 0$, $b' = 0.502$
 $L = 25.1\%$, $S = 100\%$, $H = 240^\circ$

Step 2 – Decision Tree:

Rules 1-7: Not matched ($H=240^\circ$, $L=25.1\%$, $S=100\%$)
Rule 12: $H=240^\circ \in (165, 265] \rightarrow \checkmark \text{ BLUE}$

Result: RGB(0,0,128) → BLUE $\checkmark$ (Navy correctly maps to Blue)

Example C — Light Gray RGB(180, 180, 180)

```
Step 1 - RGB → HSL:
  r' = g' = b' = 0.706 → Δ = 0 (achromatic)
  L = 70.6%, S = 0%, H = 0°

Step 2 - Decision Tree:
  Rule 1: L=70.6% < 88 → skip
  Rule 2: L=70.6% > 12 → skip
  Rule 3: S=0% < 16 → ✓ GRAY

Result: RGB(180,180,180) → GRAY ✓
```

Example D — Burgundy RGB(128, 0, 32)

```
Step 1 - RGB → HSL:
  r' = 0.502, g' = 0, b' = 0.125
  cmax = 0.502 (r'), cmin = 0, Δ = 0.502
  L = 25.1%, S = 100%
  H = ((0 - 0.125) / 0.502 + 0) × 60 = (-0.249 + 6) × 60 = 345.2°

Step 2 - Decision Tree:
  Rules 1-7: Not matched
  Rule 8: H=345.2° ≥ 345 → ✓ RED

Result: RGB(128,0,32) → RED ✓ (Burgundy correctly maps to Red)
```